Lecture 4 — Two-Dimensional Transformations

Learning outcomes

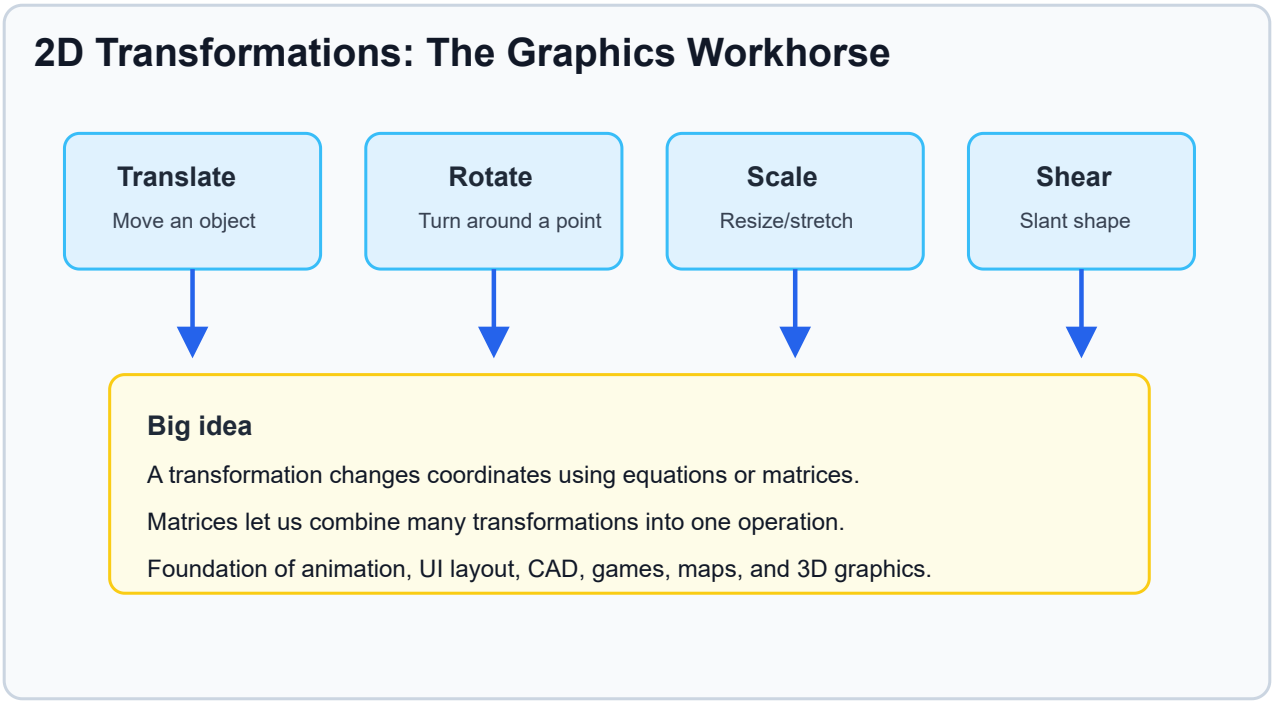
1. Explain the purpose of 2D geometric transformations.
2. Apply translation, rotation, scaling, shear, and reflection to 2D points.
3. Represent 2D transformations using matrices.
4. Explain why homogeneous coordinates are needed.
5. Combine transformations using matrix multiplication.
6. Explain why transformation order matters.
7. Rotate or scale an object about an arbitrary fixed point.
8. Distinguish between geometric transformation and coordinate transformation.
9. Explain instance transformation and its use in graphics systems.
10. Relate transformations to animation, CAD, games, maps, and user-interface design.

---

1. Why transformations are central in computer graphics

In computer graphics, we rarely draw an object only once in one fixed position. We usually want to move a character, rotate a wheel, resize an icon, mirror an object, place many copies of a model, animate objects over time, or pan and zoom a map.

All of these are examples of **transformations**.



A transformation is a rule that maps one point to another point:

```
(x, y) → (x′, y′)
```

So, a 2D object can be transformed by transforming all of its important points.

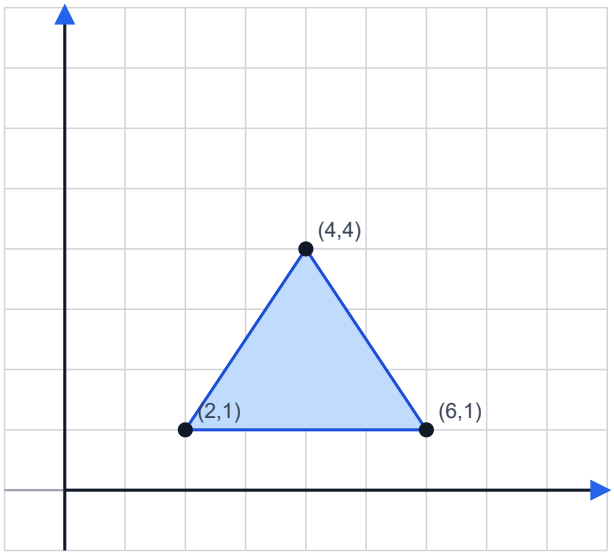
---

2. Objects as collections of points

A polygon, line drawing, icon, letter, or 2D model can be represented by points. For example, a triangle may be represented by three vertices:

```
P1 = (2, 1)
P2 = (6, 1)
P3 = (4, 4)
```

A shape as a set of coordinate points



A 2D object can be represented as vertices or sample points.

Transformations compute new coordinates:

$(x, y) \rightarrow (x', y')$

For polygons, transform the vertices. The edges follow automatically.

For images, transform pixel positions or sample from the original image.

If we transform each vertex, the whole triangle changes accordingly.

For a polygon:

```
transform vertices → redraw edges → transformed polygon
```

For a curve, we may transform control points or sampled points.

For an image, transformations are more complicated because we must resample pixels, but the coordinate idea is still central.

Trivia

In many 3D games, characters and objects are built from thousands or millions of vertices. The GPU transforms these vertices extremely fast. So the simple point transformation idea in this lecture is the same idea used in large modern graphics engines.

---

3. Translation

Translation moves every point by the same amount.

If the movement is:

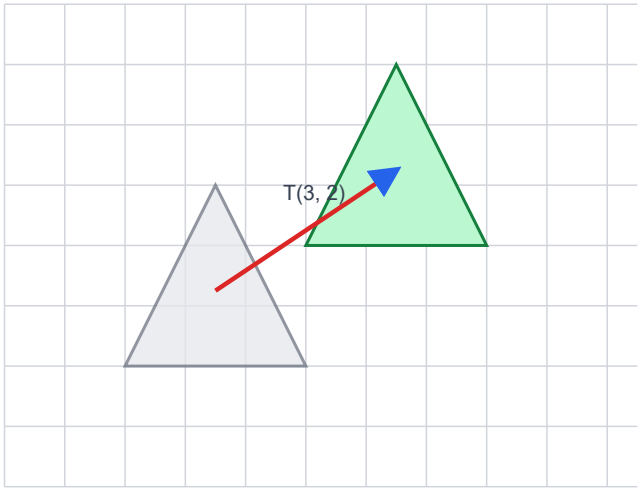
```
tₓ in the x-direction
```

```
tᵧ in the y-direction
```

then:

```
x′ = x + tₓ
y′ = y + tᵧ
```

Translation: moving every point by the same amount




```
x′ = x + tₓ
y′ = y + tᵧ

Example:
(2,1) + (3,2) = (5,3)
```

Translation preserves size, shape, and orientatior
It only changes position.

Example

Translate point `(2, 1)` by `(3, 2)`:

```
x′ = 2 + 3 = 5
y′ = 1 + 2 = 3
```

So:

```
(2, 1) → (5, 3)
```

What translation preserves

Translation preserves length, angle, shape, orientation, and area. It changes only the position.

Application

Translation is used whenever an object is moved: moving a game sprite, dragging a window, panning a map, positioning a button in a GUI, or animating a character from left to right.

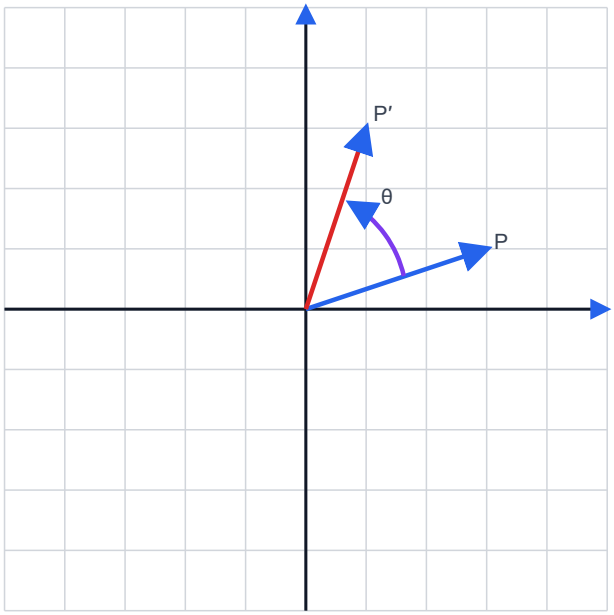
---

4. Rotation about the origin

Rotation turns a point around the origin by an angle `θ`.

For the usual mathematical coordinate system, positive rotation is counterclockwise.

Rotation about the origin




```
x′ = x cosθ − y sinθ
y′ = x sinθ + y cosθ

Matrix:
[ cosθ −sinθ ]
[ sinθ cosθ ]
```

Positive rotation is counterclockwise in the usual m

Screen coordinates may make visual direction feel

The equations are:

```
x′ = x cosθ − y sinθ
y′ = x sinθ + y cosθ
```

Matrix form:

```
[ x′ ]   [ cosθ  −sinθ ] [ x ]
[ y′ ] = [ sinθ   cosθ ] [ y ]
```

Special case: 90° rotation

For `θ = 90°`:

```
cos90° = 0
sin90° = 1
```

So:

```
x′ = −y
y′ = x
```

Example:

```
(3, 2) → (−2, 3)
```

Important coordinate warning

In mathematics, y usually increases upward. In many screen coordinate systems, y increases downward.

Therefore, a positive rotation formula may appear visually reversed on a screen unless the coordinate convention is handled carefully.

Application

Rotation is used in rotating wheels, icons, game characters, robotics simulation, map labels, and CAD components.

---

5. Scaling

Scaling changes the size of an object.

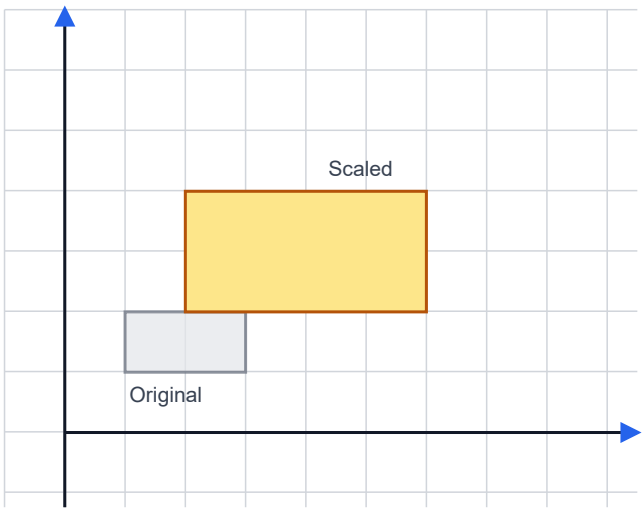
The scale factors are:

```
sₓ = scale factor in x-direction
sᵧ = scale factor in y-direction
```

The equations are:

```
x′ = sₓ x
y′ = sᵧ y
```

Scaling: resizing relative to the origin



$$x' = s_x\, x$$
$$y' = s_y\, y$$

Uniform: $s_x = s_y$

Non-uniform: $s_x \neq s_y$

Scaling about the origin also changes position unle

Matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Uniform scaling

If:

$$s_x = s_y$$

then the object grows or shrinks equally in both directions. The shape remains similar.

Non-uniform scaling

If:

$$s_x \neq s_y$$

then the object is stretched more in one direction than the other.

For example:

$$s_x = 2,\ s_y = 1$$

makes the object twice as wide but keeps the same height.

Scaling about the origin

Scaling is usually performed relative to the origin. This means the object may not only change size but also shift position.

To scale an object about its own center, we use a composite transformation.

---

6. Reflection

Reflection produces a mirror image.

Reflection about the x-axis:

```
x′ = x
y′ = −y
```

Matrix:

```
[ 1   0 ]
[ 0  −1 ]
```

Reflection about the y-axis:

```
x′ = −x
y′ = y
```

Matrix:

```
[ −1  0 ]
[  0  1 ]
```

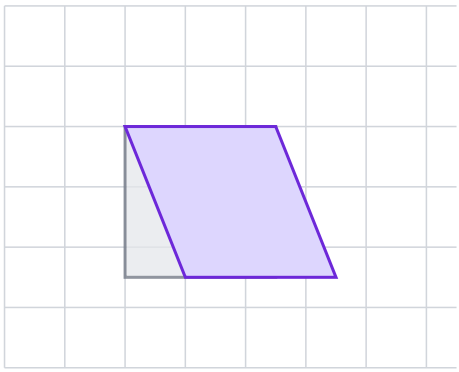
Reflection about the origin:

```
x′ = −x
y′ = −y
```

This is equivalent to a 180° rotation about the origin.

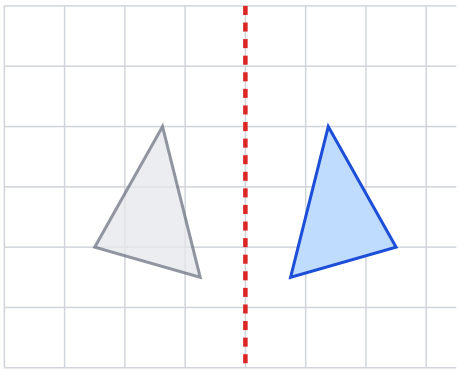
Shear and reflection

Shear: slant a shape



$x' = x + sh_x y,\ y' = y$

Reflection: mirror across an axis



Reflect y-axis: $x' = -x$

Application

Reflection is used for mirror effects, flipping sprites in 2D games, symmetric design, CAD, and image editing.

Trivia

In many 2D games, a character facing right can be made to face left simply by applying a negative horizontal scale or reflection. This avoids storing separate artwork for both directions.

---

7. Shear

Shear slants an object.

An x-shear changes x according to y:

$$x' = x + sh_x\ y$$
$$y' = y$$

Matrix:

$$\begin{bmatrix} 1 & sh_x \\ 0 & 1 \end{bmatrix}$$

A y-shear changes y according to x:

```
x′ = x
y′ = y + shᵧ x
```

Matrix:

```
[ 1   0  ]
[ shᵧ 1 ]
```

Shear is less commonly noticed than translation, rotation, and scaling, but it is important in typography, image manipulation, and geometric modeling.

---

8. The problem with ordinary 2×2 matrices

Rotation, scaling, shear, and reflection can be represented using 2×2 matrices.

But translation cannot be represented as a simple 2×2 matrix multiplication because translation requires addition:

```
x′ = x + tₓ
y′ = y + tᵧ
```

A 2×2 matrix can only compute linear combinations of x and y. It cannot add a constant offset by itself.

This creates a problem:

> We want one unified matrix system for all transformations, including translation.

The solution is **homogeneous coordinates**.

---

9. Homogeneous coordinates

In homogeneous coordinates, we represent a 2D point `(x, y)` as:

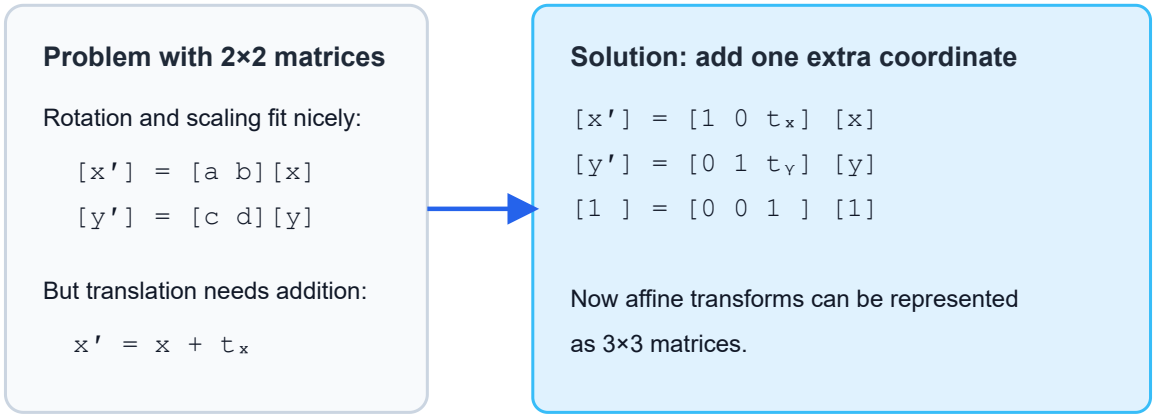
```
[x, y, 1]
```

or as a column vector:

```
[ x ]
[ y ]
[ 1 ]
```

Now transformations can be represented using 3×3 matrices.

Why homogeneous coordinates are introduced



Homogeneous coordinates become essential again in 3D projection.

Translation becomes:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

After multiplication:

$$\begin{aligned} x' &= x + t_x \\ y' &= y + t_y \end{aligned}$$

Why homogeneous coordinates are powerful

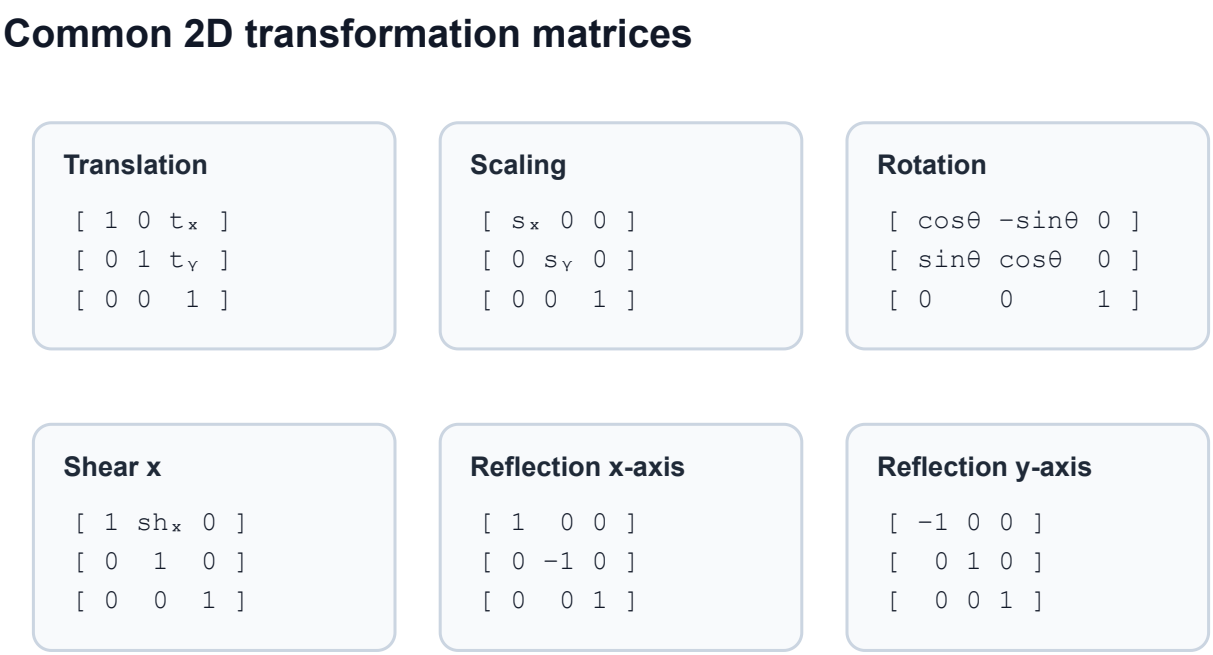
They allow us to represent all common 2D affine transformations as 3×3 matrices: translation, rotation, scaling, shear, reflection, and combinations.

Future connection

Homogeneous coordinates are even more important in 3D graphics because perspective projection also uses them.

---

10. Common 2D transformation matrices



Using column-vector convention, common matrices are:

Translation

$$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Scaling

$$\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Rotation

$$\begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \end{bmatrix}$$

```
[  0      0    1 ]
```

X-shear

```
[ 1  shₓ  0 ]
[ 0   1   0 ]
[ 0   0   1 ]
```

Reflection about x-axis

```
[ 1   0   0 ]
[ 0  −1   0 ]
[ 0   0   1 ]
```

Reflection about y-axis

```
[ −1  0   0 ]
[  0  1   0 ]
[  0  0   1 ]
```

---

11. Composite transformations

A **composite transformation** is a combination of multiple transformations.

For example:

```
scale → rotate → translate
```

Instead of applying three separate operations to every point, we can multiply the matrices together first:

```
M = T × R × S
```

Then apply the single combined matrix:

```
P′ = M × P
```

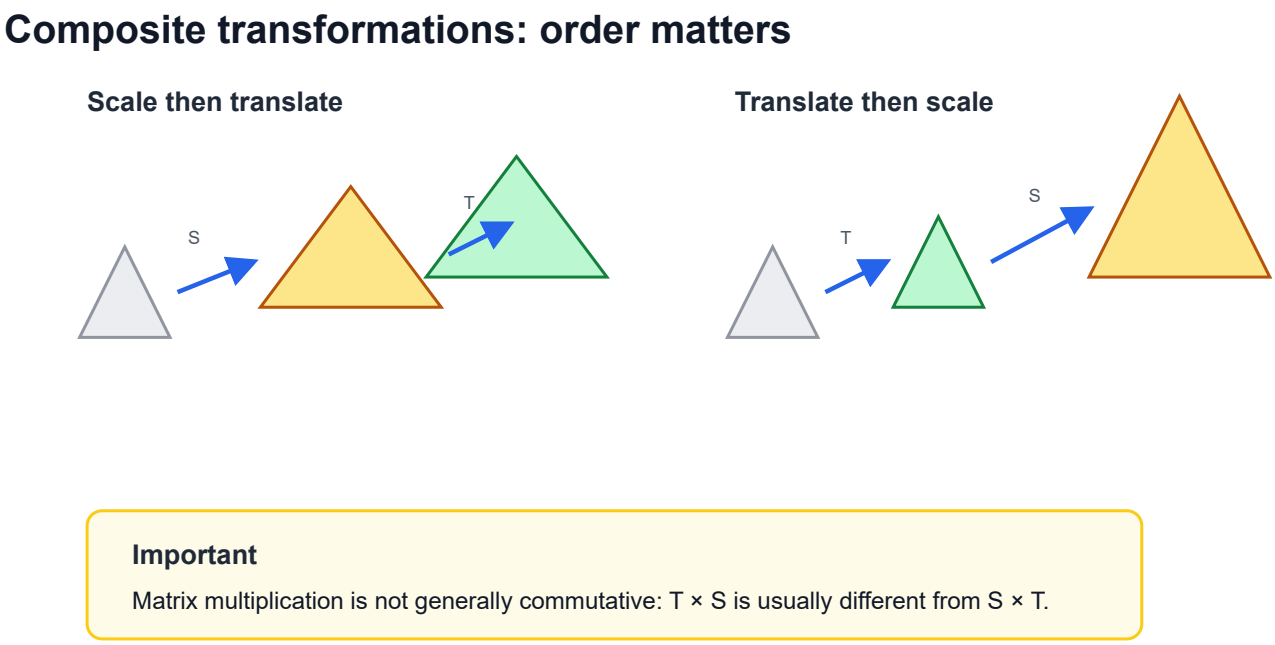
This is efficient and conceptually clean.

Order matters

Matrix multiplication is not generally commutative:

```
T × S ≠ S × T
```

So applying translation then scaling is not the same as applying scaling then translation.



> **Important**
> Matrix multiplication is not generally commutative: T × S is usually different from S × T.

Exam warning

Many students lose marks by writing the right matrices in the wrong order.

Always ask:

> What happens first to the point?

With column vectors, the rightmost matrix acts first.

For example:

```
P′ = T × R × S × P
```

means `S` acts first, then `R`, then `T`.

---

12. Rotation about an arbitrary point

The standard rotation matrix rotates around the origin.

But what if we want to rotate around a point:

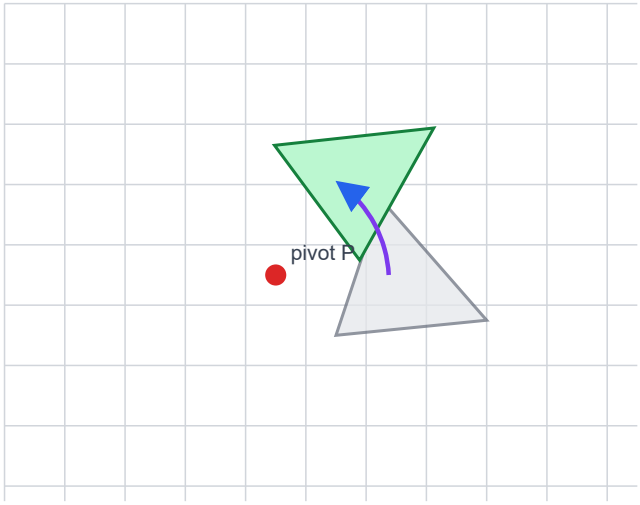
```
P = (a, b)
```

For example, a clock hand rotates around its base, not around the global origin.

The trick is:

1. translate the pivot point to the origin,
2. rotate around the origin,
3. translate back.

Rotating about an arbitrary point



1. Translate P to origin
2. Rotate about origin
3. Translate back

$M = T(P) \cdot R(\theta) \cdot T(-P)$

This “move → transform → move back” pattern is us

The composite matrix is:

```
M = T(a, b) × R(θ) × T(−a, −b)
```

With column vectors, the rightmost operation happens first:

```
P′ = T(a, b) × R(θ) × T(−a, −b) × P
```

Same pattern for scaling about a fixed point

To scale about a fixed point `(a, b)`:

```
M = T(a, b) × S(sₓ, sᵧ) × T(−a, −b)
```

Application

This pattern is used in rotating a wheel around its center, scaling an object around its center, rotating a door around its hinge, zooming into a point on a map, and animating a character limb around a joint.

---

13. Geometric transformation vs coordinate transformation

There are two ways to interpret transformation mathematics.

Geometric transformation

The coordinate system stays fixed, and the object moves.

Example:

```
Move the triangle 3 units right.
```

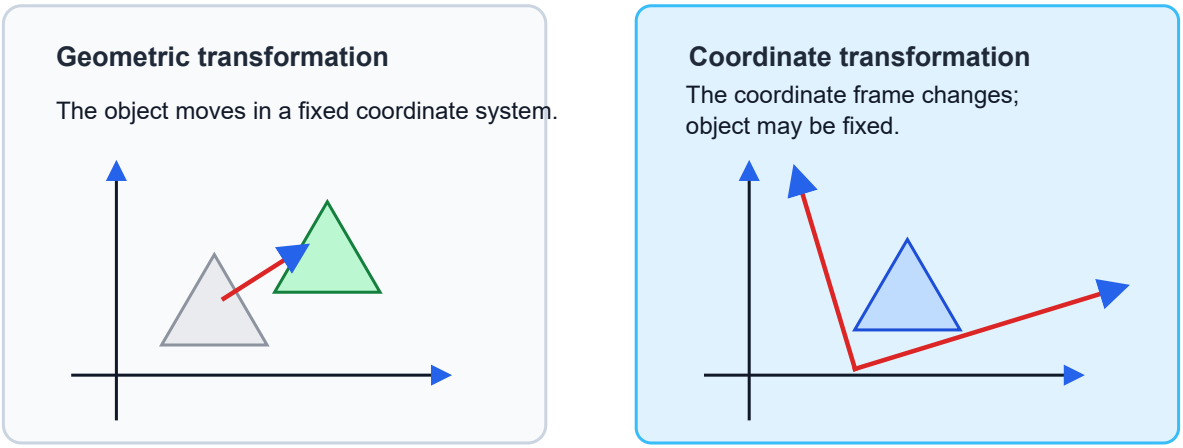
Coordinate transformation

The object may stay fixed, but the coordinate system changes.

Example:

```
Describe the same triangle using a different coordinate frame.
```

Geometric transformation vs coordinate transformation



Same algebra can describe either moving objects or changing coordinate frames.

The algebra may look similar, but the interpretation is different.

Why this matters

This distinction becomes very important in camera transformations, viewing transformations, world coordinates, local coordinates, and 3D graphics pipelines.

A camera transformation can often be understood as either moving the camera or moving the world in the opposite direction.

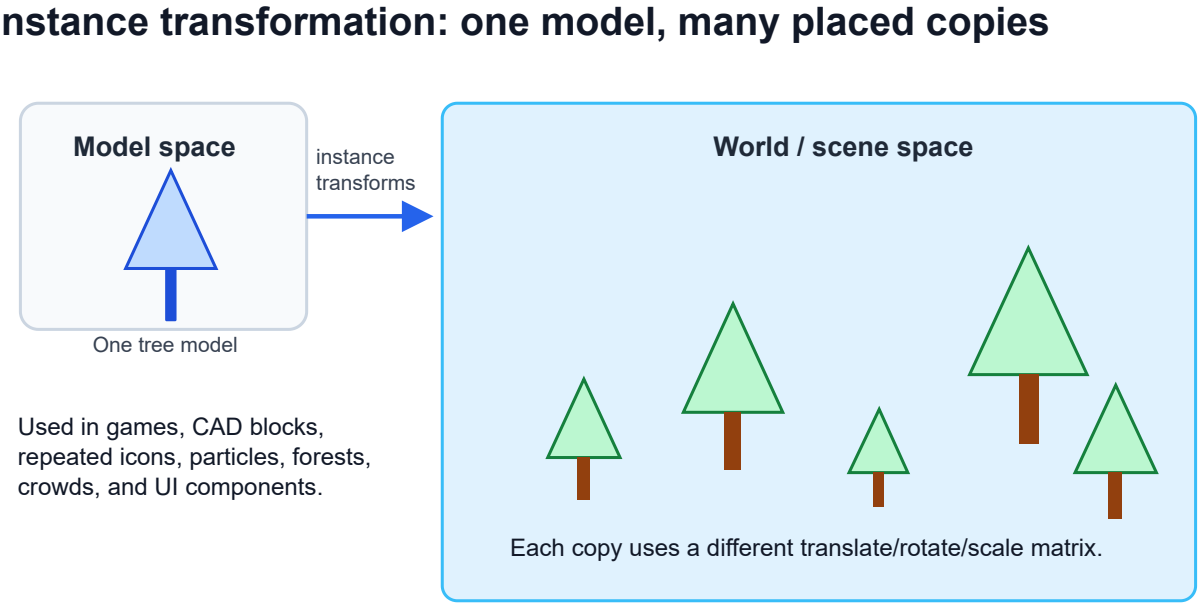
---

14. Local coordinates, world coordinates, and instance transformations

In graphics, an object is often first defined in its own local coordinate system.

For example, a tree model may be defined with its base at `(0, 0)`.

Then we place many copies of that tree into a scene using different transformations.



This is called an **instance transformation**.

Example

One tree model can be reused many times:

```
Tree 1: translate(10, 5), scale(1.0)
Tree 2: translate(20, 8), scale(0.7)
Tree 3: translate(30, 6), scale(1.3), rotate(5°)
```

The original model geometry is reused. Only the transformation matrix changes.

Why this is powerful

Instance transformation saves memory, modeling effort, rendering setup, and design time.

---

15. Transformation pipeline in 2D graphics

A typical 2D graphics pipeline may use several coordinate spaces:

```
Object/local space
      ↓ model transform
World space
      ↓ view transform
Camera/view space
```

```
      ↓ viewport transform
Screen space
```

In simple 2D programs, some of these stages may be merged. But the conceptual separation is useful.

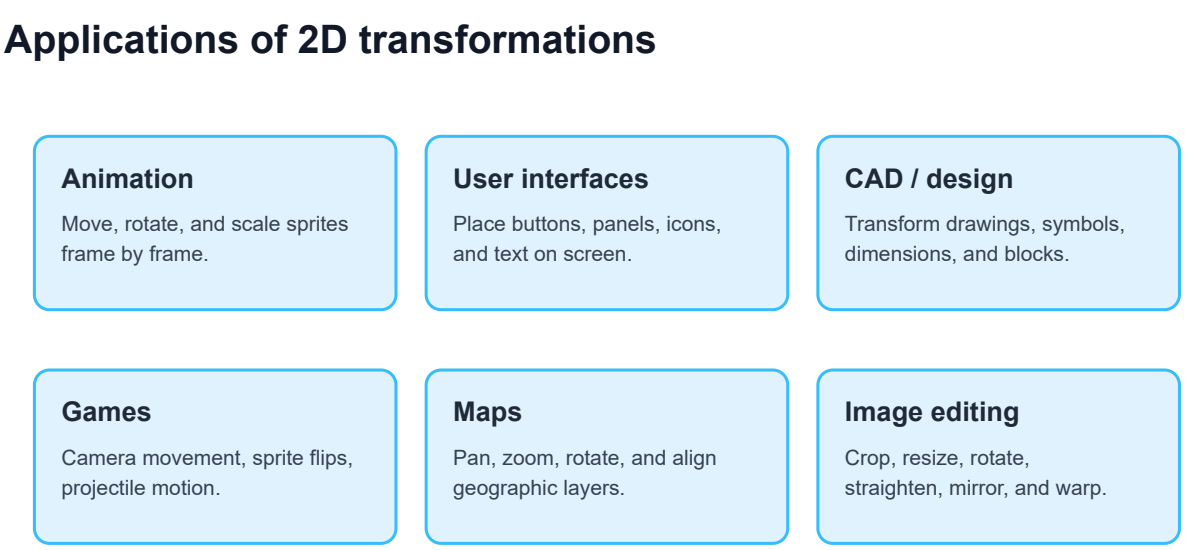
Connection to future lectures

This same pipeline idea becomes even more important in 3D graphics, where we use model transformation, view transformation, projection transformation, and viewport transformation.

So Lecture 4 is a bridge between 2D graphics and 3D graphics.

---

16. Applications of 2D transformations



Trivia: “transform” is one of the most frequently used ideas in all of computer graphics.

Transformations appear everywhere:

Animation

An object moves because its transformation changes over time.

```
position(t) = position0 + velocity × t
```

User interface design

Buttons, icons, panels, and text are placed using transformations and layout rules.

CAD and engineering drawings

Parts, blocks, dimensions, and symbols are copied, rotated, scaled, and aligned.

Games

Sprites are translated, rotated, scaled, flipped, and grouped.

Maps

Pan and zoom are transformations. Rotating a map is also a transformation.

Image editing

Crop, rotate, mirror, resize, straighten, and warp operations all rely on geometric transformations.

---

17. Worked examples

Example 1: Translation

Translate `(4, 2)` by `(3, −1)`.

```
x′ = 4 + 3 = 7
y′ = 2 − 1 = 1
```

Answer:

```
(4, 2) → (7, 1)
```

Example 2: Scaling

Scale `(3, 4)` by:

```
sₓ = 2
sᵧ = 3
```

Then:

```
x′ = 2 × 3 = 6
y′ = 3 × 4 = 12
```

Answer:

```
(3, 4) → (6, 12)
```

Example 3: Rotation by 90°

Rotate `(5, 2)` by 90° counterclockwise about the origin.

For 90°:

```
x′ = −y
y′ = x
```

So:

```
(5, 2) → (−2, 5)
```

Example 4: Composite transformation

A point `P = (1, 2)` is first scaled by `(2, 2)` and then translated by `(3, 4)`.

First scaling:

```
(1, 2) → (2, 4)
```

Then translation:

```
(2, 4) → (5, 8)
```

Final answer:

```
P′ = (5, 8)
```

If we reverse the order, we get:

First translation:

```
(1, 2) → (4, 6)
```

Then scaling:

```
(4, 6) → (8, 12)
```

Different result.

This demonstrates that order matters.

---

18. Common mistakes

1. Thinking transformation order does not matter.
2. Forgetting that the rightmost matrix acts first when using column vectors.
3. Rotating about the origin when the problem asks for rotation about another point.
4. Scaling about the origin and being surprised that the object moves.
5. Confusing screen coordinates with mathematical coordinates.
6. Mixing row-vector and column-vector conventions.
7. Forgetting to use homogeneous coordinates for translation.
8. Applying transformation only to one vertex of a polygon instead of all vertices.

---

19. Short exam-style questions

1. What is the difference between translation and scaling?
2. Write the 3×3 homogeneous matrix for translation by (t_x, t_y).
3. Write the rotation matrix for angle θ.
4. Why do we need homogeneous coordinates?
5. Why does transformation order matter?
6. How do you rotate an object about a point (a, b)?
7. What is the difference between geometric transformation and coordinate transformation?
8. What is an instance transformation?
9. Give two applications of 2D transformations in computer graphics.
10. What happens when $s_x = -1$ and $s_y = 1$?

---

20. Practice problems

Problem 1

Translate triangle vertices:

```
A(1, 1), B(4, 1), C(2, 3)
```

by:

```
T(3, 2)
```

Find the new vertices.

Problem 2

Scale the same triangle by:

```
sₓ = 2, sᵧ = 3
```

about the origin.

Problem 3

Rotate point `(2, 5)` by 90° counterclockwise about the origin.

Problem 4

A point `(1, 1)` is first translated by `(2, 0)` and then scaled by `(3, 3)`. Find the final point.

Then reverse the order and compare the result.

Problem 5

Explain the steps needed to scale an object about its own center instead of the origin.

---

21. Lecture summary

Lecture 4 summary

Translation	Add an offset to every point.
Rotation	Turn points around the origin or a chosen pivot.
Scaling	Resize relative to the origin or a fixed point.
Shear / Reflection	Slant or mirror a shape.
Homogeneous coordinates	Represent affine transforms using 3×3 matrices.
Composition	Combine many transforms, but order matters.
Instance transforms	Reuse one model many times in a scene.

Big idea: transformations are the language of movement, placement, layout, and animation.

The big takeaways are:

- Transformations map old coordinates to new coordinates.
- Translation moves objects.
- Rotation turns objects.
- Scaling resizes objects.
- Reflection mirrors objects.
- Shear slants objects.
- Homogeneous coordinates allow all affine transformations to be represented as 3×3 matrices.
- Composite transformations combine multiple operations.
- Transformation order matters.
- Arbitrary pivot transformations use the pattern: translate to origin, transform, translate back.
- Instance transformations allow one model to appear many times in a scene.

---

22. Final trivia box

- The word “transform” appears everywhere in graphics APIs, game engines, animation tools, SVG, CSS, CAD software, and 3D rendering systems.
- CSS transforms such as `translate`, `rotate`, `scale`, and `skew` are direct practical uses of the same ideas from this lecture.
- In a modern GPU pipeline, millions of vertices can be transformed every frame.

- The simple 3×3 matrix from this lecture is the 2D cousin of the 4×4 transformation matrices used in 3D graphics.